Cupcake Rapport

Deltagere

Jakob Svennekjær Barken
jakob.barken@gmail.com
AlteredProject | Hold B

Bertram Friis Møller
Bertramfriis1@gmail.com
BertramFM | Hold B

Indholdsfortegnelse

Indledning 1
Baggrund 1
Teknologi 1
Krav 2
Domæne Model & ERD 3
Domæne model 3
ERD 4
Project board 4
Navigations diagram 5
Særlige forhold 6
Status på implementation 6
Proces 6

Indledning

Denne rapport er lavet til en fagfælle på 2. semester datamatikeruddannelsen.

Projektet omhandler udviklingen af et website for virksomheden Olsker Cupcakes. Virksomheden ønsker at få udviklet en digital løsning der tillader online bestilling af deres produkter.

Opgaven tager udgangspunkt i en halvfærdig mockup, der fungerer som visuel skitse af en mulig forside, derudover er der givet et antal 'User-Stories' der giver os nogle kravspecifikationer til udarbejdningen af denne digitale løsning. Det færdige produkt skal udvikles i Java med Javalin og Thymeleaf, derudover gøres der brug af en Postgres database, samt HTML og CSS til frontend. Projektet indebærer derfor både teknisk implementering og kravafklaring.

Baggrund

Olsker Cupcakes er et bornholmsk bageri der sælger cupcakes, efter et besøg fra nogle Københavnske hipstere har de givet følgende krav til den digitale løsning:

- Man skal kunne bestille og betale for cupcakes m. valgfri bund og top.
- Man skal kunne oprette en profil så man kan gemme sine ordrer
- Der skal være administrator kontoer som kan se og administrere alle kunder og deres ordrer som f.eks. at indsætte beløber ind på kunders kontoer eller slette en kundes ordrer
- En kunde skal i en indkøbskurv kunne se og slette ordre linjer fra deres ordrer hvis de fortryder et valg af cupcakes, samt se den samlede pris for kundens ordrer.

Målet for denne digitale løsning er at gøre det nemmere for Olsker Cupcakes at håndtere ordrer, samt gøre det muligt for kunder at bestille ordrer uden nødvendigvis at skulle ned i butikken bare for at lægge en bestilling.

Teknologi

IntelliJ IDEA – V.2025.2.4

Javalin – V.6.7.0

Thymeleaf – V.3.1.3

Junit – V.5.12.0

Postgres – V.42.7.

Docker – V.4.60.0

SDK – Amazon Corretto V.17.0.18

Krav

Denne digitale løsning skal gøre det muligt for kunder at kunne bestille og betale for ordrer af cupcakes online sådan at de senere kan køre forbi butikken i Olsker og hente deres ordre.

US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken i Olsker og hente min ordre.

US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.

US-3: Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.

US-4: Som kunde kan jeg se mine valgte ordrelinier i en indkøbskurv, så jeg kan se den samlede pris.

US-5: Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mockup'en).

US-6: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

US-7: Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

US-8: Som kunde kan jeg fjerne en ordrelinie fra min indkøbskurv, så jeg kan justere min ordre.

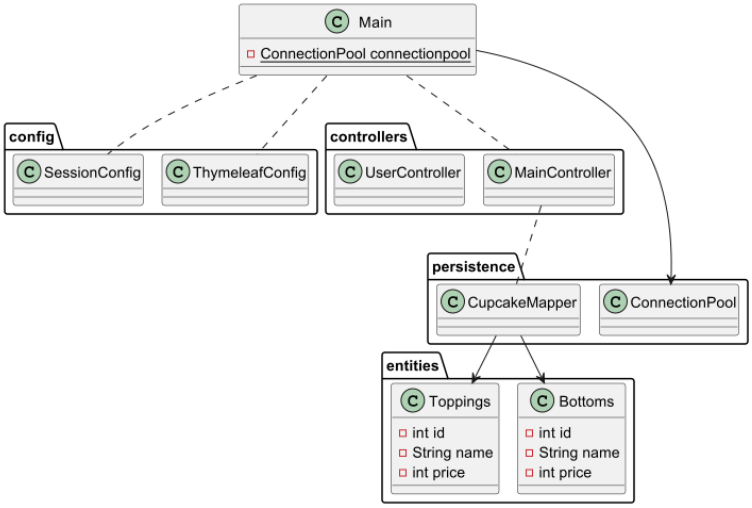
US-9: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde udgyldige ordrer. F.eks. hvis kunden aldrig har betalt.

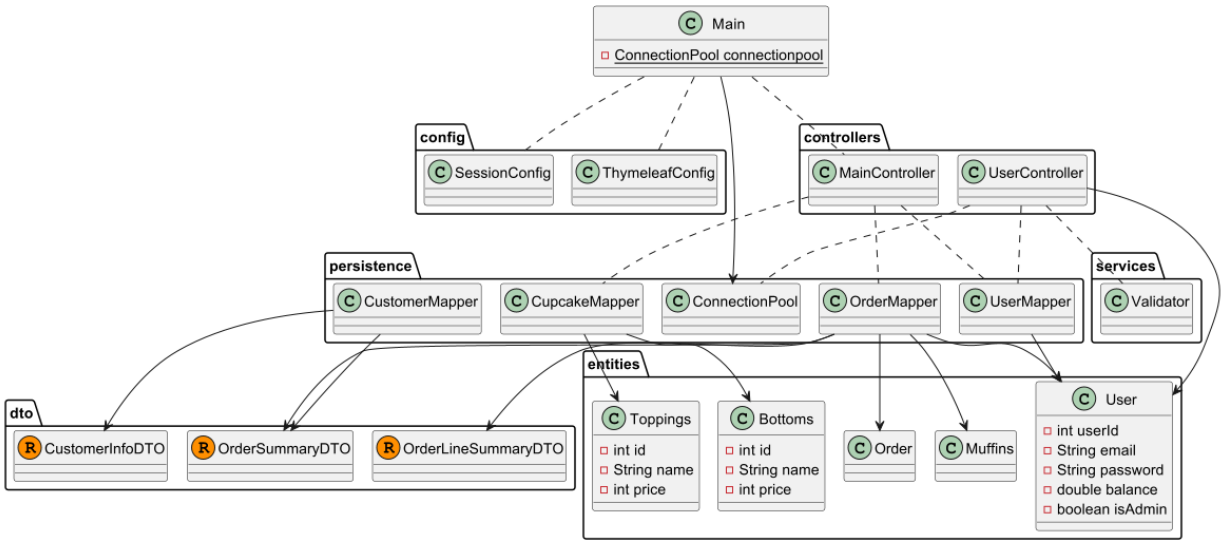
Domæne Model & ERD

Nedenstående billeder viser en oversigt over Domæne model af Java klasserne, samt en ERD af databasen.

Domæne model

Nedenstående billeder viser første version af domænemodel imod den færdige domænemodel, her ses at vi senere valgte at implementere DTO'er til håndtering af hvilken data vi giver videre fra databasen, dette valg tog vi da vi både skulle have information fra flere tabeller på 1 gang, samt undgå at give brugeres loginoplysninger med når man bare skal se en brugers kredit balance og e-mail. Derudover var der ting vi havde overset at tilføje såsom User og nogle mappere.

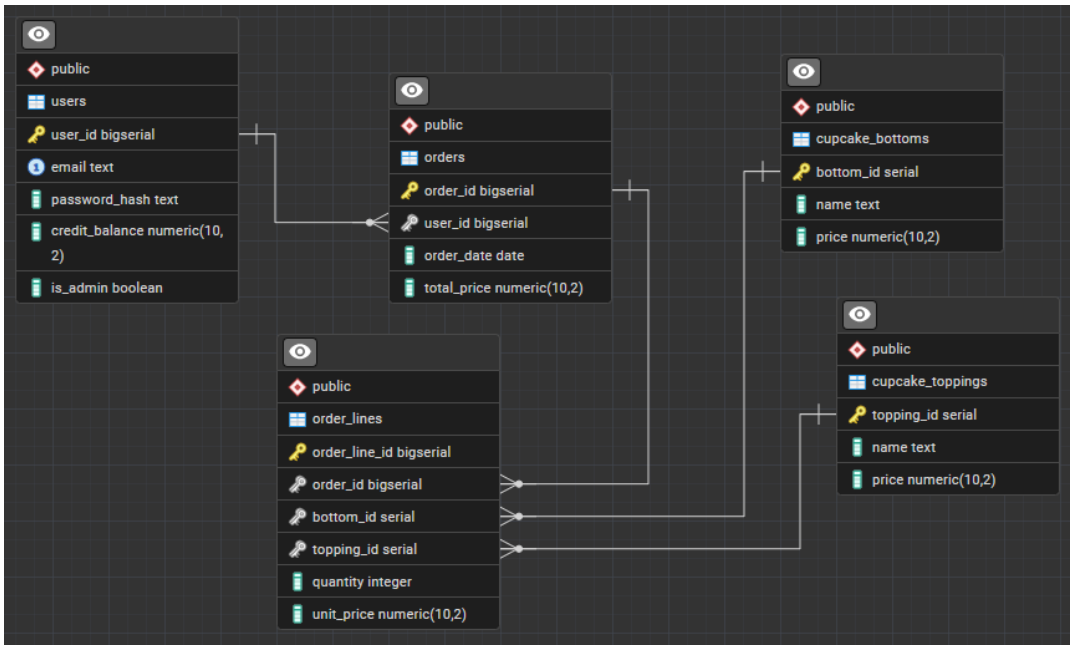




ERD

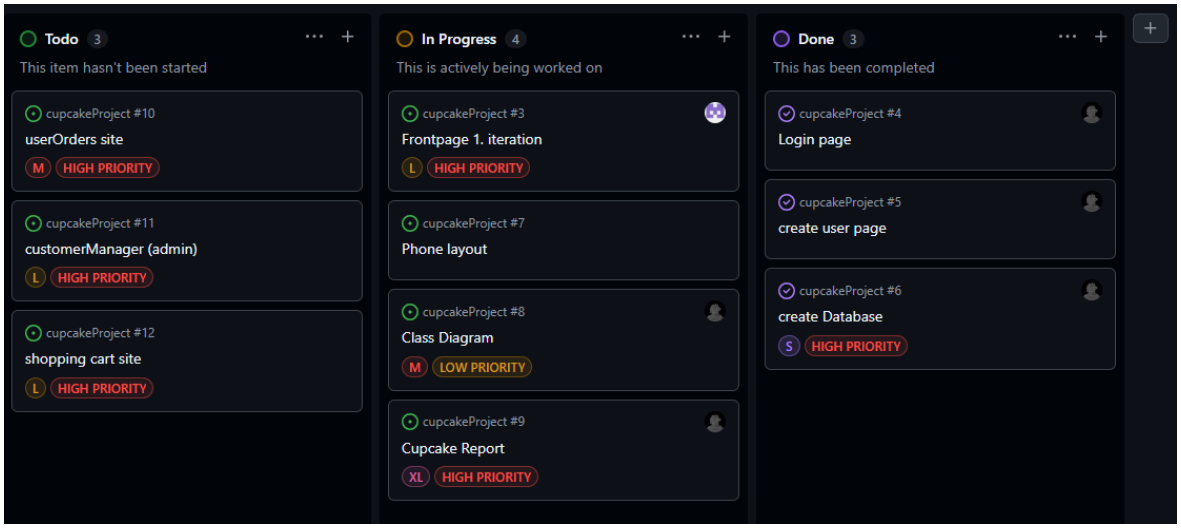
Databasen var noget af det første vi satte op, på baggrund af at det ville give et godt overblik over den data vi har, derudover gjorde det nemmere for os når vi lavede backend delen da databasen var klar med det samme til at modtage dataen.

Senere tilføjede vi unit_price og total_price, for at tillade Olsker cupcakes at kunne ændre på priserne uden at det påvirker gamle allerede-betalte ordrer i databasen.



Project board

Undervejs har vi gjort brug af et projekt board på GitHub til at have overblik og holde styr på: hvad vi har lavet, hvad vi mangler og om vi er bagud eller ej.

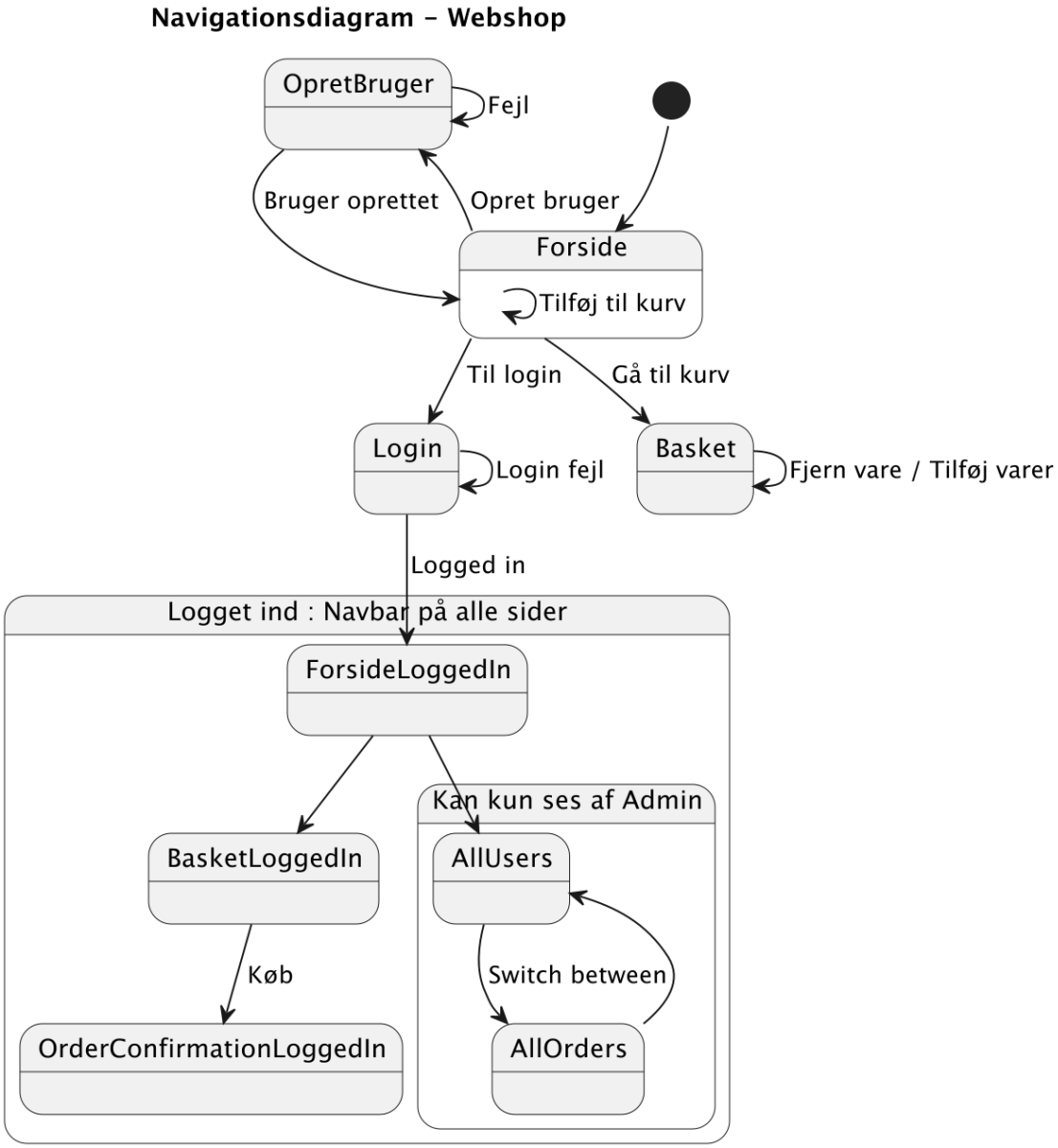


Navigations diagram

Når man først kommer til hjemmeside og man ikke er logget ind, kan man se login, registrer bruger, kurv og firmalogo (tilbage til forsid).

Når man er logget ind som almindelig bruger kan man yderlige få lov til at købe/bestille en muffin ordre.

Hvis man er logget ind som administrator, kan man nu se “Kunder” i navbar. Navbaren følger med på alle sider.



Særlige forhold

Vi har gjort brug af Javalin's Session handler til at gemme brugerens login og hvilke cupcakes de har gemt i kurven på tværs alle sider på websitet. Derudover har vi sikret at websitet ikke crasher ved at have exceptions og validatorer til at håndtere eventuelle inputs fra brugeren. Denne Validator er også med til at lave sikkerhed på hjemmesiden, da en brugers password skal opfylde visse krav før de kan oprette deres konto. Hvilke brugere der har administrator privilegier håndteres direkte i databasen, så kun folk med adgang til databasen kan ændre om en bruger er administrator eller ej.

Status på implementation

Vi har i vores forståelse af US 1-9, fået implementeret dem alle til en rimelig tilfredsstillelse. Vi har fået stylet alle sider så det passer til telefon først og skaleret op til computerskærm ved brug af @media.

Dog manglede vi at inkorporere at brugerens kredit bliver trukket fra deres konto vha. CRUD i databasen.

Proces

Vores plan for udarbejdningen af opgaven var at begynde med at få en generel idé om hvordan vi gerne ville have websitet til at se ud ved at lave en Figma designfil, samt finde et passende farvetema til hjemmesiden og lave en domænemodel med de klasser vi forventede at ville have med, vi fandt undervejs ud af at vi manglede en del klasser i forhold til hvad vi havde forventet, især da vi endte med at gøre brug af DTO'er.

Vi begyndte dagene med at lave forventninger for hvad vi håbede at nå den dag, derefter arbejdede vi fokuseret fra 10-15 hver dag med pauser indimellem, til sidst på dagen lavede vi opsummering omkring hvad vi mangler at lave, for at holde styr på om vi er bagud eller ej, i forhold til vores deadline.

Noget vi kunne have gjort, var at når vi stødte på en fejl eller et problem, i stedet for bare at gå i gang med at fikse det, så lige tage note af hvad problemet er, og efter vi har løst det, hvordan vi fik det løst.